

RAI: Value Transfer with Account-Local Consensus

Optional Proof Supplement

Anonymous Author(s)

This supplement expands the mathematical arguments in the main paper. It does not change the protocol or add assumptions to its guarantees. All statements use the main paper’s single-support account rule, static per-committee fault bound, cryptographic assumptions, persistent voting records, placement checks, and data-retention rules. Termination additionally uses its stated availability and progress-window assumptions. These are design proofs, not a mechanized verification or an implementation audit.

1 Voting invariants

Fix a session, an epoch, and a committee of size

$$N = 3f + 2p + 1, \quad q = N - f - p = 2f + p + 1, \quad r = f + p + 1.$$

An account domain also fixes the account and account slot. An epoch-election domain instead fixes the shared election slot and the committee role. The two roles have separate records, even for an identity belonging to both committees. A first vote also counts as notarization support; counting a first vote and later support by the same identity twice is forbidden.

The following local facts are persistent. A correct identity issues at most one first vote per domain. In an account domain, that first vote is its only notarization support; it may normal-final-vote only for that first-voted block after the block becomes complete and eligible. In an epoch-election domain, it retains the supported-value set and the existing second-look rules; a normal final vote requires that this set contain no conflicting value. In either domain, signing the final vote atomically makes the domain terminal, so no later callback can issue further support. Checkpoint settlement never clears these records. In the epoch election, completion or shared skip may stop voting sooner; stopping cannot weaken the invariants.

Lemma 1.1 (Account NC uniqueness and normal exclusion). *In an account domain, at most one block has an NC. In either kind of domain, a valid normal final certificate for h excludes a notarization certificate for $h' \neq h$ in the same domain.*

Proof. Two account NCs have signer sets intersecting in at least $2q - N = f + 1$ identities. A correct identity in that intersection would have supported both blocks, contrary to single-support voting. Thus account NCs are unique.

For normal exclusion in either kind of domain, let F and V be the final and notarization signer sets, each of size q . Then

$$|F \cap V| \geq 2q - N = f + 1.$$

At least one identity in the intersection is correct. It could not support h' before final-voting for h , because the final-vote guard forbids this. It could not support h' afterward, because the final-vote transition is terminal. Both time orders are impossible. This proof does not require any validator to have assembled the final certificate when the conflicting vote would be sent. $\square$

Lemma 1.2 (Fast exclusion). *A valid fast certificate for h excludes a notarization certificate for any different value in the same domain, including an epoch-election timeout.*

Proof. Let $f' \leq f$ be the actual number of faulty members. Among $N - p$ first signers for h , at least $N - p - f'$ are correct. There are $N - f'$ correct members altogether, so at most p can first-vote for any other value. Thus every different value has at most $f' + p \leq f + p < r$ first signers. This count applies in both kinds of domain.

In an account domain, the fast certificate supplies at least q first votes for h , hence an NC; account NC uniqueness excludes any conflicting NC. It remains to consider an epoch-election domain. In any receiver's first-vote map, the number of entries not supporting h is at most $f' + p < r$. This remains true if Byzantine identities equivocate: the receiver records only one first vote per identity.

No other value can trigger a correct second look. Its first votes alone are fewer than q , since $f + p < 2f + p + 1$. For the epoch-election split test, write c_h for the recorded count of h , taking $c_h = 0$ when absent. Then

$$all - max \leq all - c_h \leq f + p < r.$$

Thus the split test cannot create correct timeout support either. A timer-based timeout first vote belongs to the already bounded set of first votes not supporting h . Since every correct non-timeout vote requires complete valid data and proofs, an invalid proposal cannot be used to bypass these counts. There is no other rule that can supply a conflicting notarization quorum. $\square$

Lemma 1.3 (Compatible account prefixes). *Within a fixed epoch, any two settled current-epoch account notarizations, and hence any two eligible account finalizations, have compatible selected ancestor prefixes.*

Proof. Both placements settle against the same predecessor checkpoint. Let its maximum represented depth for the account be d . Every new voted block is above d ; every unresolved ancestor above d needed a current-epoch certificate before a descendant could be voted on.

Suppose notarized tips B and C conflict, with B no deeper than C . At equal depths, they contradict account NC uniqueness. Otherwise the selected ancestry of C contains a different block at B 's depth with a current-epoch NC, again contradicting uniqueness. Both account finality forms supply notarization evidence, so this also covers finalized tips. Compatible tips have compatible ancestors, including their selected paths through the predecessor ledger.

This argument also covers votes issued during overlap. The retained validation closure already contains the required current-epoch ancestors, and eligibility checks the resulting placement against the decided predecessor before making the certificate final. $\square$

2 Frozen reports and checkpoint recovery

Fix a closed predecessor S_{e-1} and a selection Q of $N - f$ usable reports for epoch e . A usable report has an authenticated, finite, semantically valid evidence closure. The closure includes required ancestors and finalized dependencies, not only the reporter's tips. Certificate claims in that closure are fixed by the signed semantic records. An equivalent set of witness signatures can verify an existing claim but cannot add a claim to the frozen report.

A placement is certified-visible when this closure represents its settled current-epoch notarization or stronger eligible finality. A finality annotation establishes its notarization for the definitions below. A placement is final-visible when the closure supplies eligible final evidence for it or a finalized descendant containing it. Raw early evidence that fails settlement is in neither class.

Let $M_Q(h)$ count selected identities whose own residual first or notarization vote supports h . Candidate inclusion is

$$h \text{ is certified-visible} \quad \vee \quad M_Q(h) \geq 1.$$

This is not a finality predicate. Required ancestry is included as well, inherited unresolved branches are retained, and every candidate is branch-valid. A residual candidate need not yet have its own notarization certificate.

At a new position (a, v) , define

$$\begin{aligned}\mathcal{V}_Q(a, v) &= \{h : h \text{ is certified-visible at } (a, v)\}, \\ c_Q(h) &= |\{i \in Q : G_i \text{ contains } i\text{'s own residual First}(h)\}|, \\ \mathcal{A}_Q(a, v) &= \{h : c_Q(h) \geq r\}.\end{aligned}$$

Account NC uniqueness gives $|\mathcal{V}_Q(a, v)| \leq 1$, but residual candidates and members of $\mathcal{A}_Q(a, v)$ may still conflict. The recovery target is

$$\text{Protect}_Q(a, v) = \begin{cases} \{h\}, & \mathcal{V}_Q(a, v) = \{h\}, \\ \{h\}, & \mathcal{V}_Q(a, v) = \emptyset \text{ and } \mathcal{A}_Q(a, v) = \{h\}, \\ \emptyset, & \text{otherwise.} \end{cases}$$

These sets are computed before pruning. Mandatory targets comprise inherited finality, final-visible blocks, required final dependencies, and these recovery targets. Their selected ancestors are mandatory too. In particular, a weaker residual conflict cannot turn a unique visible certificate into an unresolved checkpoint fork.

Lemma 2.1 (A possible normal certificate is preserved). *Every selected report quorum finalizes an eligible block for which a normal final certificate can be formed, even when that certificate is first assembled after the reports or checkpoint decision.*

Proof. Let F be the q final signers. The selection and F share at least

$$|F \cap Q| - f \geq q + (N - f) - N - f = p + 1$$

correct identities. Every such identity final-voted before signing its report, because reporting permanently stops old-epoch signing. It already had the block's notarization certificate when it final-voted. Report normalization must preserve that status, or stronger finality for a descendant whose selected prefix includes the block. A reporter cannot summarize away that fact merely because a chosen certificate witness omits its own signature.

Stronger finality directly makes the block mandatory. Otherwise the block belongs to $\mathcal{V}_Q$. Account NC uniqueness prevents any conflicting certificate, so this set is a singleton. The first recovery rule makes the block and its selected ancestors final.

Correct selected signers cannot cast missing old-epoch votes after reporting. Unselected correct identities may still vote, and Byzantine signatures may be withheld, but neither event changes the intersection bound or removes the already represented certificate. $\square$

Lemma 2.2 (A possible fast certificate is preserved). *Every selected report quorum finalizes an eligible block for which a fast certificate can be formed, with the same allowance for late assembly.*

Proof. Let A be its $N - p$ first signers. At least

$$|A \cap Q| - f \geq (N - p) + (N - f) - N - f = f + p + 1 = r$$

selected first signers are correct. If any of them has summarized its vote by a notarization certificate or stronger status, that status is in the frozen closure. Account NC uniqueness then gives the singleton-certificate case or direct final-visible inclusion.

Otherwise each of these r identities retains its own first vote in its residual object, and $c_Q(h) \geq r$. The first-vote count in Lemma 1.2 bounds all first signers for every conflicting block in the domain by $f + p < r$. Hence no other block can enter $\mathcal{A}_Q$. Either $\mathcal{V}_Q = \{h\}$ or $\mathcal{V}_Q$ is empty and $\mathcal{A}_Q = \{h\}$. The corresponding recovery rule finalizes h .

Eligibility matters here. For a certificate that is eligible against the fixed predecessor, correct supporters retained the required placement and ancestry before voting. Settlement therefore does not discard the supports used in the intersection. An ineligible raw tally has no such finality obligation and is not covered by the statement. $\square$

Lemma 2.3 (Compatible mandatory targets). *For usable reports against a valid predecessor, mandatory targets have compatible account histories.*

Proof. Consider two conflicting targets on one account, with the shallower target at depth v . If the targets have equal depth, two final-visible targets contradict account-prefix compatibility. A final-visible target excludes a different certified target by account NC uniqueness, and rules out residual-first recovery by making $\mathcal{V}_Q$ nonempty. Two recovery targets cannot be different: the selection rules require the relevant set to be a singleton.

Now let the other target be deeper. Its evidence closure contains a current-epoch notarization for its different ancestor at depth v . For a certified or final-visible deeper target, this follows directly from its complete selected ancestry. For a residual-first deeper target, its threshold $r > f$ includes a correct reporter who had and retained this ancestry before voting. Early voting cannot omit the evidence: unresolved ancestry above the eventually decided frontier had to be certified in the same epoch.

If the shallower target is final-visible or selected by unique-certificate recovery, this conflicting ancestor contradicts account NC uniqueness. If it is selected by residual-first recovery, the ancestor contradicts the required emptiness of $\mathcal{V}_Q$. These cases cover both types of target in either depth order.

Inherited finalized history and earlier-epoch final dependencies are respected by placement validation. Current-epoch final dependencies are themselves final-visible. Thus all mandatory selected prefixes are compatible. Their finite, acyclic dependency proofs permit branch replay without assuming the checkpoint being constructed. $\square$

Lemma 2.4 (Finite deterministic construction). *The construction in the main paper terminates and returns one checkpoint for its fixed usable inputs.*

Proof. Canonical input expansion produces a finite parent-closed candidate forest. The preceding lemma makes the mandatory closure compatible. Mark that closure final and remove conflicts. At each later round, consider all positions having exactly one remaining candidate. Two such choices cannot conflict: if a deeper choice selected a different ancestor at a shallower chosen position, parent closure would supply a second survivor there. Thus each simultaneous round is compatible.

No round inserts a new candidate. Finality only grows and the remaining candidate set only shrinks. A nonterminal round marks at least one previously nonfinal block final or removes at least one block. The process therefore stops after finitely many rounds. This proves termination for the specified simultaneous procedure; it does not assert confluence for arbitrary interleavings of different pruning rules.

All newly final prefixes are replayed into a fresh copy of the predecessor application state, in the specified dependency-respecting canonical order. This candidate computation changes no live balance; only a joint decision installs the checkpoint. Parent order and each block's signed operation order are preserved. An external receive proof points to an already final send, so independent account operations do not acquire an artificial choice of spend order. Canonical roots, fixed status claims, deterministic committee derivation, and deterministic replay give one output, independently of a validator's live unselected evidence. $\square$

3 The joint epoch election

The election has one shared placement tree and two committee-specific vote pools. A non-genesis placement copies its parent's payload. Its parent is jointly complete in a strictly earlier slot. Every intervening slot has a shared skip proof. Shared skip means a timeout certificate in either committee, or non-timeout notarization certificates for different placements across the two committees.

Lemma 3.1 (A joint decision cannot be skipped). *A placement with matching fast certificates in both committees, or matching normal final certificates in both, excludes every shared skip at its slot.*

Proof. Each committee has local final evidence for the decided placement. Local exclusion rules out its timeout certificate. For cross-committee notarizations of $x \neq y$, any proposed jointly finalized placement differs from at least one of x, y . That committee's local exclusion rules out the proposed finality. Thus none of the skip alternatives can coexist with the joint decision. A one-sided final certificate does not establish this fact for the other committee and is not an epoch decision. $\square$

Theorem 3.2 (One checkpoint per epoch). *Any two joint decisions in an epoch have the same payload and derived state.*

Proof. Order two decisions by slot number. Let B be the decision at the smaller slot v . The decision at the greater slot has a finite chain of strictly decreasing parent slots ending at genesis. The chain cannot cross v without a node there: crossing would require a skip proof for v , contrary to the preceding lemma. At slot v , local exclusion makes the chain's node B . Every descendant copies its non-genesis parent's payload. Consequently the decision at the greater slot carries B 's payload, regardless of when either certificate is assembled. Decisions in the same slot cannot conflict by local exclusion. Deterministic checkpoint construction then gives one state. $\square$

Lemma 3.3 (A prepared slot advances). *In a sufficiently long prepared synchronous interval with $N - f$ responsive correct identities in each committee, a shared slot eventually supplies joint completeness or shared skip evidence.*

Proof. Examine one committee. A correct identity that already ended local voting relays the complete certificate causing its transition. If it stopped on a shared skip, the desired shared outcome already exists. Otherwise suppose no correct identity has yet terminated. All $N - f$ responsive correct identities eventually first-vote, with timeout supplying a first vote when no valid proposal arrives.

If a non-timeout value has at least r correct first voters, their complete data and proofs reach every correct identity. Every still-voting correct identity can then validate and second-look that value. The resulting correct support is enough for notarization because $N - f \geq q$. If some identity terminates earlier instead, its relayed certificate supplies a local outcome.

If no non-timeout value has r correct first voters, their first votes alone satisfy

$$all - max \geq (N - f) - (r - 1) = r.$$

Adding a new map entry increases *all* by one and increases *max* by at most one. Byzantine entries cannot decrease the difference. Every remaining correct identity can therefore support timeout, giving a timeout certificate. Termination during this reasoning again supplies a local outcome instead of removing a voter without evidence.

Apply the argument in both committees. Either has a timeout, yielding shared skip; or both have a non-timeout notarization. Matching values give joint completeness and different values give the cross-committee skip. Correct identities continue relaying evidence after local termination, so the shared controller does not wait for additional votes from them. $\square$

4 Safety across epochs and application replay

Theorem 4.1 (Irreversible transfers). *Under the main paper’s safety assumptions, correct validators have compatible finalized account histories. Finalized balances are nonnegative and each send is received at most once.*

Proof. Induct over closed epochs from genesis. Joint agreement gives a unique predecessor for each epoch. Account-prefix compatibility handles explicit and ancestral finalization in its account elections. Report recovery preserves every possible eligible final certificate, including certificates assembled after a handoff. Checkpoint construction makes its additional choices compatible and preserves inherited finality.

During overlap, successor first votes and the notarization support they provide have no application effect. They are not private vetoes over the closing election. Successor finality is enabled only after the predecessor checkpoint is known and the entire placement settles above its maximum represented account frontier. An incompatible early tally cannot become spendable. Persistent vote records prevent reusing the same domain after settlement. Hence overlap cannot break the epoch induction.

For application safety, induct over finalization events. Every event replays the complete selected unfinalized prefix from its finalized anchor, using a single temporary balance and received-send set per account. An overdraft or repeated receive is rejected before any effect is committed. A receive must identify an irreversible finalized send addressed to that account. Compatible finalized histories therefore cannot contain two successful receives of that send, even when its first receive was in an unfinalized ancestor at validation time. An applied-operation record shared by online finalization and checkpoint installation prevents a second application of the same live operation. Pure candidate-state computation does not execute live effects. No nonfinal block contributes an application effect or affects committee derivation. $\square$

5 Termination and its scope

Reconstruction. A correct report has finite immutable contents. Correct validators continuously update their canonical local inventories and roots as epoch- e records arrive. By eventual gossip and finite admitted work, the inventories converge and therefore have the same root. Under fair retries and service, a requester and a correct reporter eventually share that source root and reconstruct the reporter’s frozen signed inventory. The residual object and evidence closure are fetched separately, and root verification and semantic checks finish in finite time. An unavailable Byzantine report does not block a selection because $N - f$ correct reports suffice. Validators that vote on a proposal also retain its usable contents.

Epoch closure. A progress window must cover catch-up, preparation of admissible report selections, a full leader cycle, and voting. With entry skew at most κ_E , combined preparation and validation budget τ_E , and prepared-message budget δ , the main paper requires

$$\frac{\Delta_E}{1 + \rho} > \kappa_E + \tau_E + \delta.$$

The left side is the shortest real-time duration of a correct local timeout. A correct leader’s valid proposal therefore arrives before the earliest first timeout in a fresh prepared slot. Every correct first vote supports it. Byzantine first votes alone are fewer than r , so they cannot trigger a conflicting second look or the split timeout rule. Each committee gets notarization and then a normal final certificate; fast finality is not needed.

The slot-progress lemma passes earlier stalled slots. Round-robin leadership has a finite gap between correct leaders in the known union of committees. The stipulated overlap of a long enough synchronous interval with that leader cycle supplies the successful slot. A one-sided final certificate cannot prevent it,

because only shared parent and skip evidence constrain the history. Closing one epoch removes the guard deferring the next boundary, giving ordered epoch closure by induction.

This is not a proof of progress from network synchrony alone. The $N - f$ responsive-correct-member condition is needed by the split-vote argument, and the report quorum also has size $N - f$, not q . Nor does a fixed timer guarantee bounded reconstruction for arbitrarily large Byzantine inputs. Finite admitted work and the explicit preparation bound are essential premises.

Account continuations. Consider the next block of a persistently submitted, finite, valid uncontested continuation. In a ready common-open interval after predecessor closure, each correct validator can first-vote the block or has already done so. An old conflicting owner signature would violate this premise; merely stopping new equivocation is insufficient. Prepared parents and send dependencies permit all checks to finish.

All correct support concerns the same block, so at least $N - f \geq q$ first votes give notarization. Those first voters can each final-vote for that same block, giving normal finalization without any account timer.

If the epoch boundary interrupts an attempt, completed finality is preserved by reports. A sole represented candidate may instead finalize at the checkpoint. An omitted candidate can be retried in a new epoch without changing its hash. The recurring ready-window premise guarantees an attempt long enough to complete, not just arbitrarily many interrupted attempts. A fresh uncontested child of a retained maximum-depth tip uses the same reasoning and finalizes its selected unresolved ancestors. Induct over the finite continuation and use epoch closure to checkpoint each result.

Prepared latency. After a prepared block is broadcast, correct first votes are sent by δ , notarization and normal final votes are available by 2δ , and normal finality by 3δ . If $N - p$ correct first voters respond in that interval, fast finality is available by 2δ . Processing an available first-vote action before a newly enabled terminal final-vote action is needed for this fast-path statement. Both bounds concern prepared communication phases, not end-to-end time with arbitrary reconstruction or client delay.

As a sufficient account budget, let openings differ by at most κ and let readiness take at most R after the last opening. Then

$$\frac{D}{1 + \rho} > \kappa + R + 3\delta$$

leaves a normal voting window before the earliest boundary. A boundary deferred by an older closing epoch only adds time. The theorem does not guarantee progress for a passive recipient, an account that continues to have contested new positions, a permanently unavailable quorum, or sustained overload.

Local signing bound. In an account domain, a correct validator issues at most one first vote, its only notarization support, and at most one normal final vote for that same block. It makes no account second looks. This gives at most two distinct local signatures per account domain, not a bound on retransmissions, gossip traffic, certificate validation, state size, or data-transfer latency. Epoch-election second looks and their voting rules are unchanged.